OrderCache — Exam Sheet

Cover this, reproduce on a blank page, then check page 2. Re-do tomorrow.

Drill A — the matching formula

Write, from memory:

1. The **one-sentence hook**.
2. The **formula in symbols**.
3. The **four-line code loop**.
4. Why the $-s_c$ term is there (one sentence).
5. For **Buy**200(A), **Sell**150(A), **Buy**40(B), **Sell**90(C): the answer, and *why it's that and not* $\min(B, S)$.

Drill B — the test suite

For each of the six methods, list every test you'd write **and the one-line bug it traps**. Aim for the four layers:

1. examples (the spec's case)
2. edges — one trap per bug
3. relationships (round trips, order)
4. oracle (random vs. reference) + perf

Target: 14+ tests, each with its bug named.

Drill C — the design (say it in 30s)

From memory: name the **four data structures**, what each makes fast, the **complexity** of all six methods, and **why the two helpers** exist.

Answer Key

Drill A answers

1. Hook: "Everyone shops the market *minus themselves*; add up what they grab; never beat the smaller pile."

2. Formula: $M = \min\Big(\sum_c \min(b_c,\ S - s_c),\ \min(B, S)\Big)$

3. Loop: `reach=0; for c: reach += min(c.buy, totalSell - c.sell); return min(reach, min(B,S));`

4. $-s_c$**:** a firm may not use its *own* sells, so its allowed pool is all sells minus its own — that enforces the different-company rule.

5. Answer $= 130$. B=240, S=240. A: $\min(200, 240-150)$=90; B: $\min(40, 240)$=40; C: 0. Sum $= 130 \le \min(B, S)$=240, so the *reach* binds. It's not 240 because A's big buy may only reach the 90 of sells that aren't A's own.

Drill B answers — test / bug it traps

L1 README example (2700)	formula plain wrong
L2 self-only → 0	firm matches itself
one cross match	counting own-company sells
dominant company → 150	using naive $\min(B, S)$
only-one-side → 0	missing empty-side guard
invalid ignored	no validation (empty id, qty 0, bad side)
duplicate id ignored	second add corrupts
unknown lookup/cancel no-op	missing graceful guard (throws)
`minQty` boundary (=)	`>` vs `>=` off-by-one
overflow near `UINT_MAX`	32-bit accumulator
L3 add→cancel restores match	aggregate not decremented on cancel
cancel-for-user no trace	index desync / ghosts
insertion order irrelevant	hidden order-dependent state
L4 random vs. oracle (seed)	any matching bug, unseen cases
random churn vs. oracle	maintenance bug under add/cancel
million-order timing	an $O(n)$ scan in the query

Drill C answers

Structures: `orders_` (id→Order, source of truth; makes cancelById/getAll fast) · `ordersByUser_` (user→ids; cancelForUser) · `ordersBySecurity_` (sec→ids; the per-sec cancel) · `securityTotals_` (sec→{B,S,per-company b/s}; matching).

Complexity: add/cancel $O(1)$; bulk cancels $O(k)$; matching $O(\#\text{companies})$; getAll $O(n)$.

Why two helpers (`insertOrderInternal`/`removeOrderInternal`): every write funnels through them, so the four structures update in *one place* and can't desync.